

Evaluation Metrics

What will you learn:

- *The different evaluation metrics for deep learning object detection.*
- *What is Average Precision?*
- *What is IoU (Intersection over Union)?*
- *mAP (mean Average Precision) evaluation metric.*

Precision in Machine Learning

Precision is not a deep learning or object detection concept. It is a simple machine learning concept. Now, what is precision?

- **Precision**: It tells us, *out of all the predictions that our model says are positive, how many are actually positive*? Or, *What proportions of all* the positive predictions are actually positive?

$$Precision = \frac{TP}{TP + FP}$$

- In the above formula, TP is True Positive, and FP is False Positive.

Precision in Machine Learning

Precision is not a deep learning or object detection concept. It is a simple machine learning concept. Now, what is precision?

- ***Recall***: *Out of all the positive cases, how many positive cases does the model identify??*

$$Recall = \frac{TP}{TP + FN}$$

- In the above formula, TP is True Positive, and FN is False Nagative.

Precision in Machine Learning

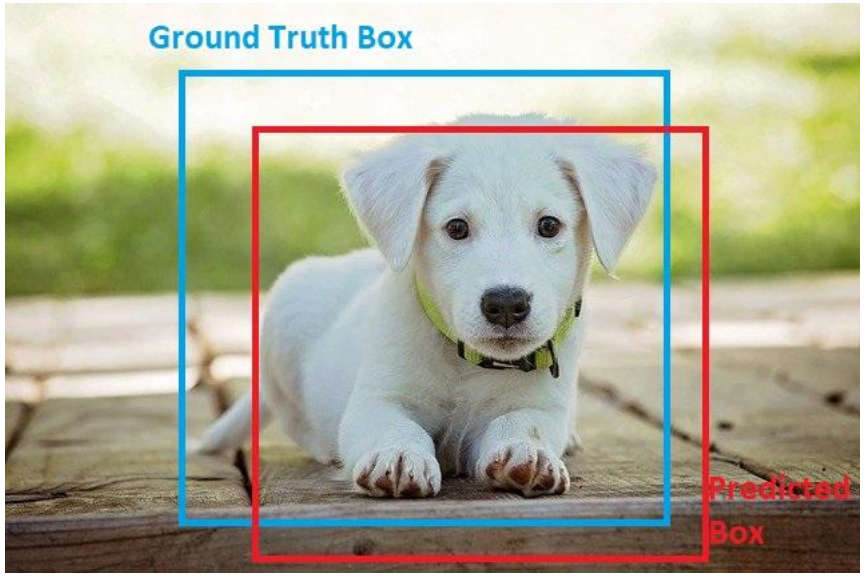
Example: Suppose that we have a bunch of images. We have to identity whether the image is of a dog (positive case) or not. Then:

$$Precision = \frac{TP}{total\ positive\ results\ by\ the\ model}$$

$$Recall = \frac{TP}{total\ number\ of\ dog\ images}$$

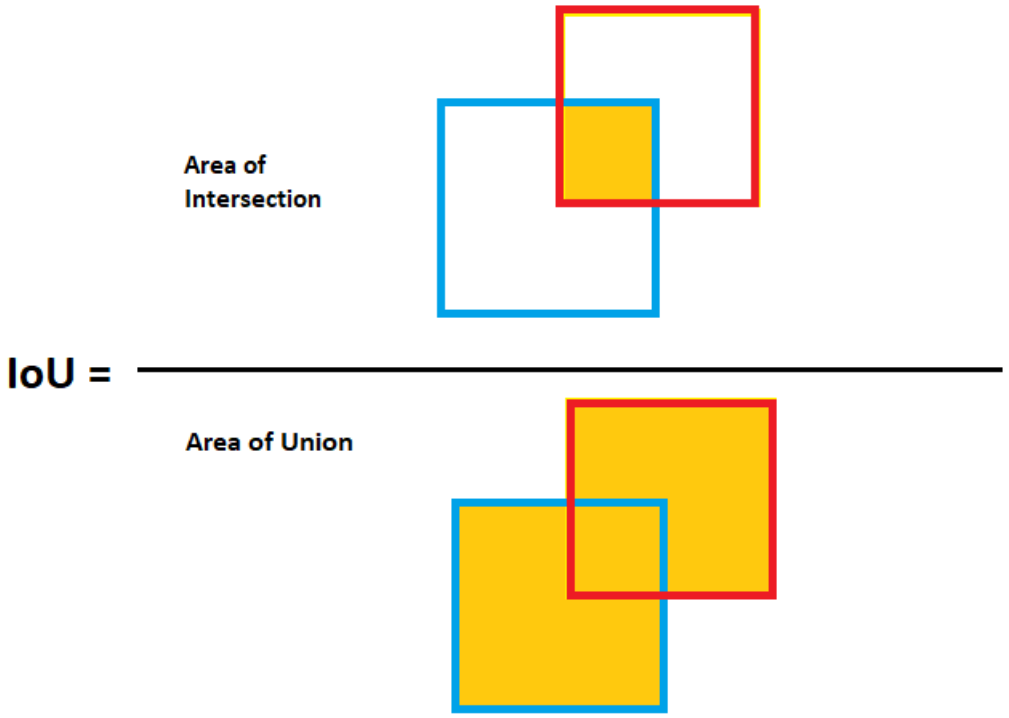
Intersection Over Union (IoU)

- IoU gives the overlap between two bounding boxes.
- In object detection, it gives the overlap between the ground truth bounding box and the predicted bounding box.



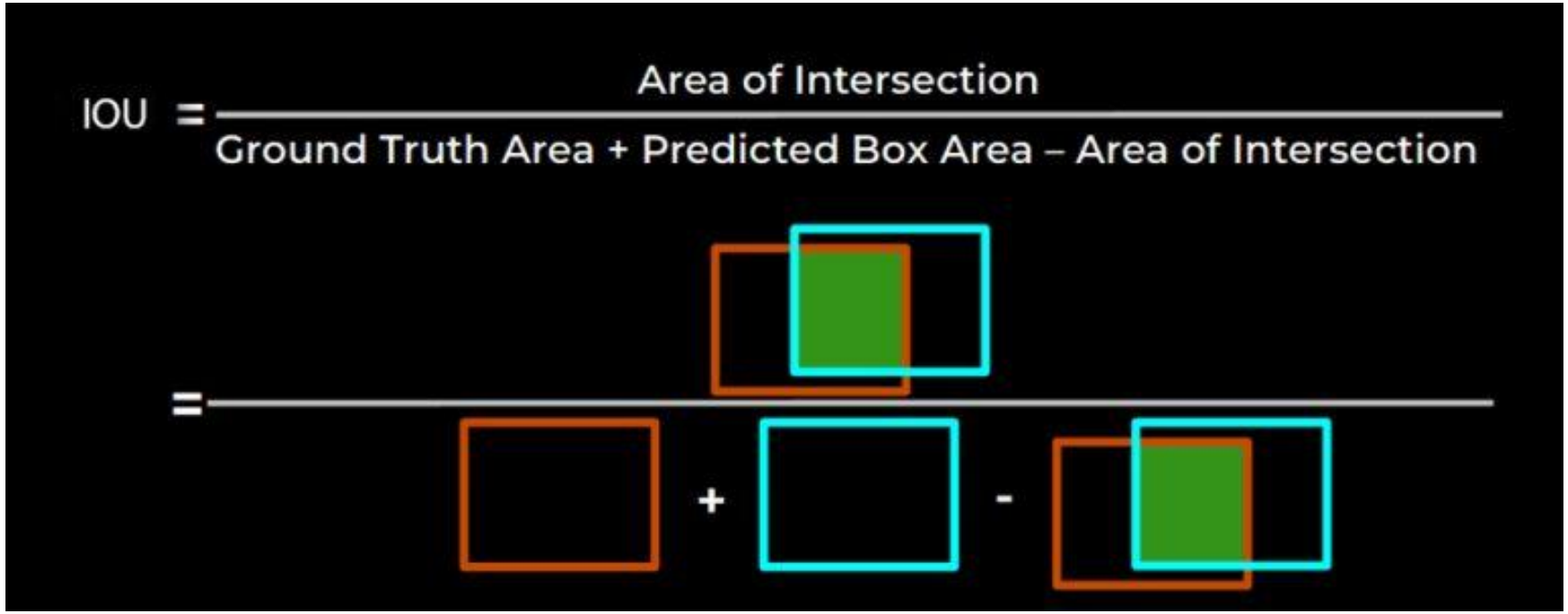
Intersection Over Union (IoU)

- Basically, IoU is the overlap (intersection) of the predicted and ground truth boxes divided by the union of the predicted and ground truth boxes..

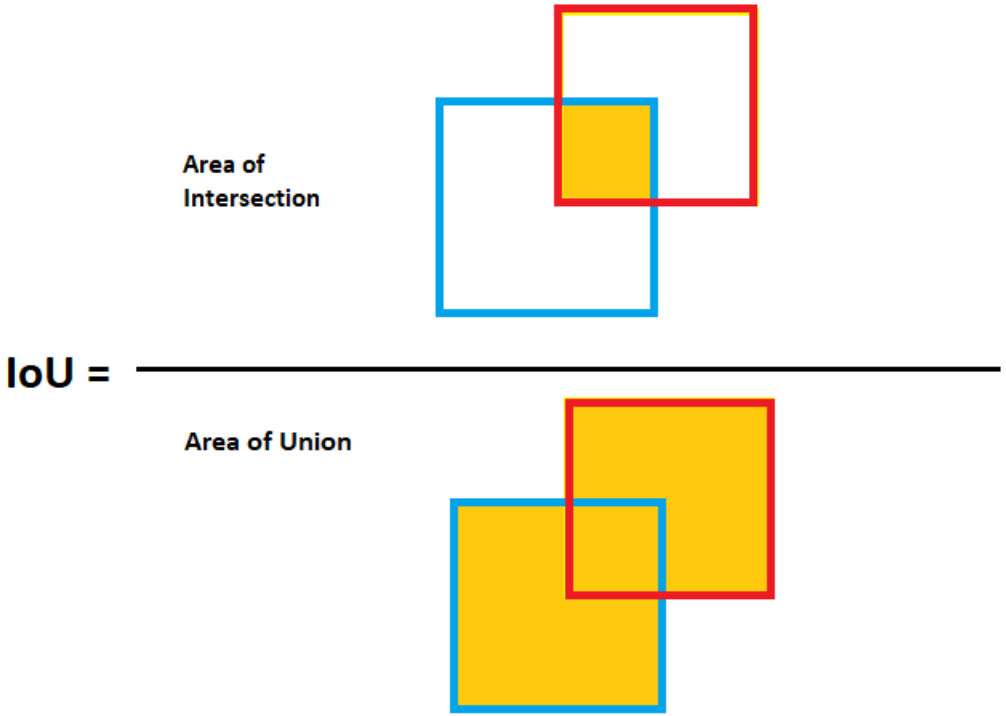


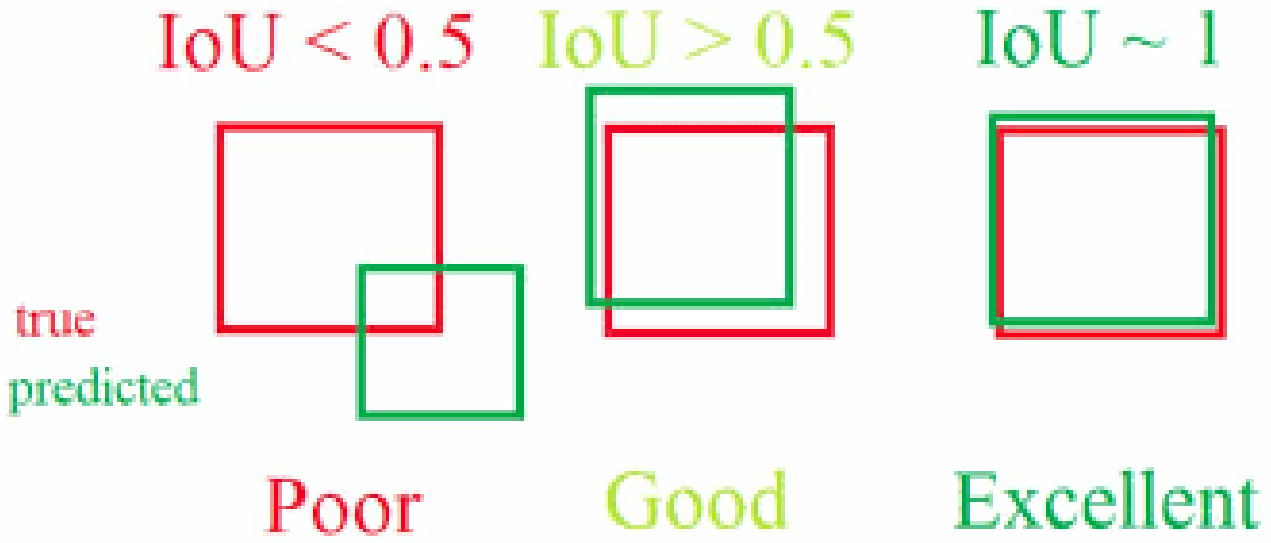
Intersection Over Union (IoU)

- Basically, IoU is the overlap (intersection) of the predicted and ground truth boxes divided by the union of the predicted and ground truth boxes..



Intersection Over Union (IoU)





Where are you getting the ground-truth examples from?

when training your own object detector (such as the HOG + Linear SVM method), you need a dataset. This dataset should be broken into (at least) two groups:

- A **training set** used for training your object detector.
- A **testing set** for evaluating your object detector.

You may also have a **validation set** used to tune the hyperparameters of your model.

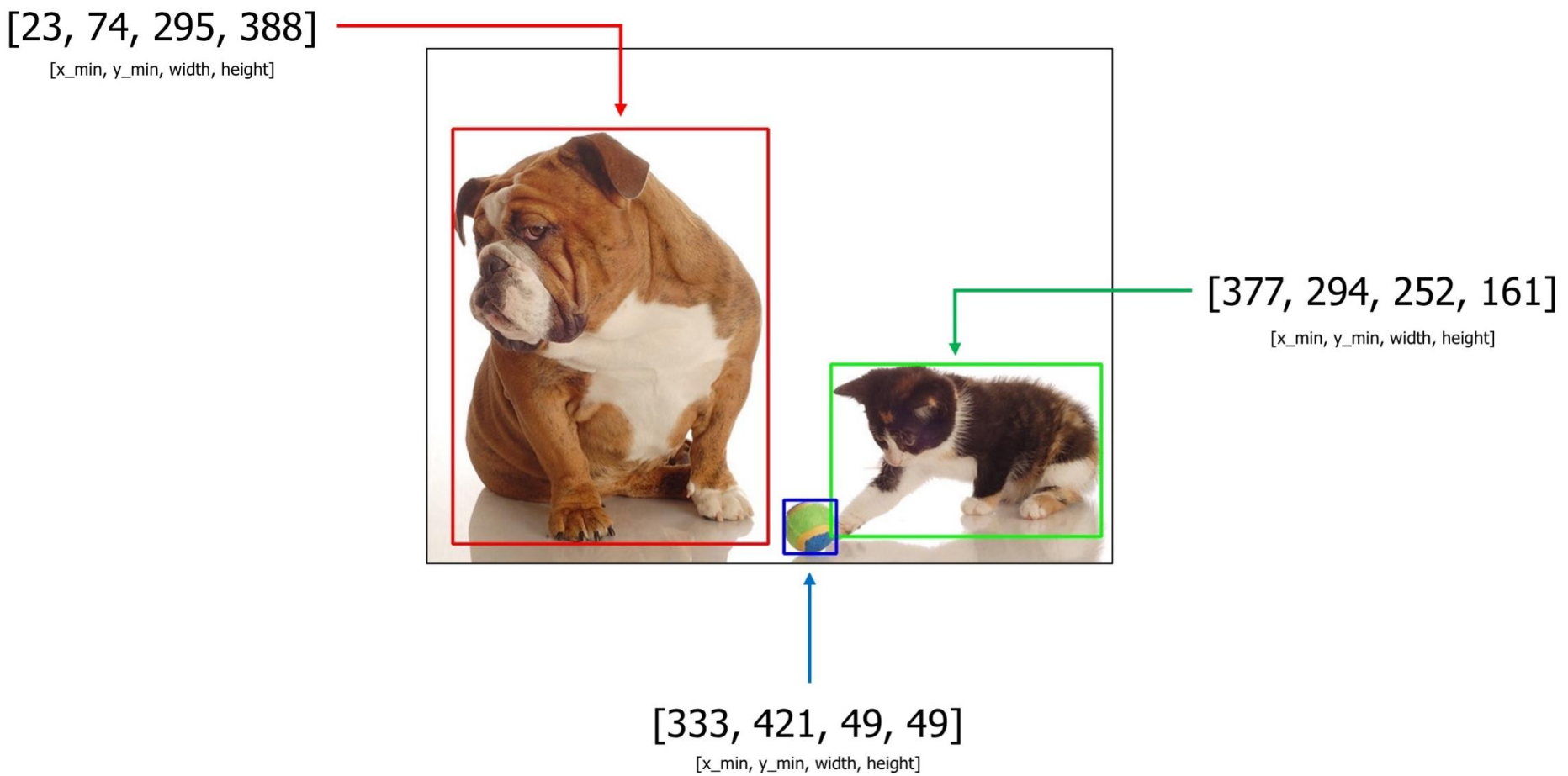
Where are you getting the ground-truth examples from?

Both the training and testing set will consist of:

- The actual images themselves.
- The bounding boxes associated with the object(s) in the image. The bounding boxes are simply the (x, y)-coordinates of the object in the image.

The **bounding boxes** for the training and testing sets are ***hand labeled*** and hence why we call them the "ground-truth".

Where are you getting the ground-truth examples from?



An example image with 3 bounding boxes from the COCO dataset

Determining Results from IoU

How do we determine at which IoU value there is an object inside the predicted bounding box?

- For most datasets and even competitions like MS COCO and PASCAL VOC, an **IoU of 0.5 is good enough**.
- This means that if the IoU between the ground truth bounding boxes and the predicted boxes is 0.5, then we conclude that there is an object inside the box. This is know an ***minimum IoU threshold***.

Determining Results from IoU

If IoU >= 0.5 then it is a true positive.

This means that there is an object and we have detected it.

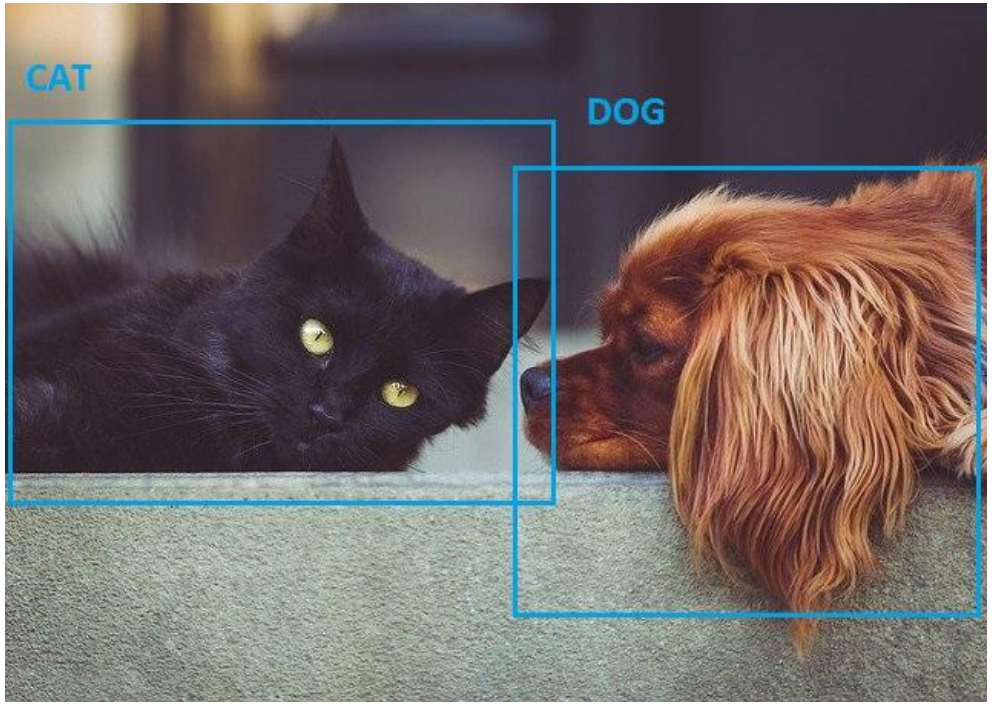


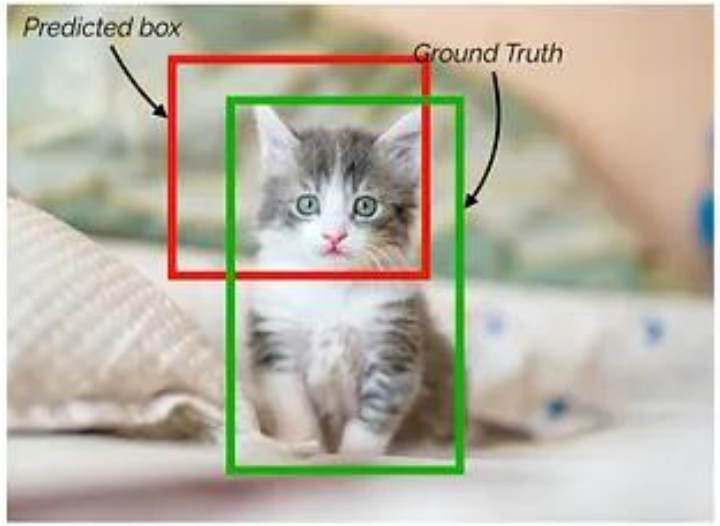
Figure 3. True Positive case in deep learning object detection.

Determining Results from IoU

If IoU < 0.5 then it is a false positive.

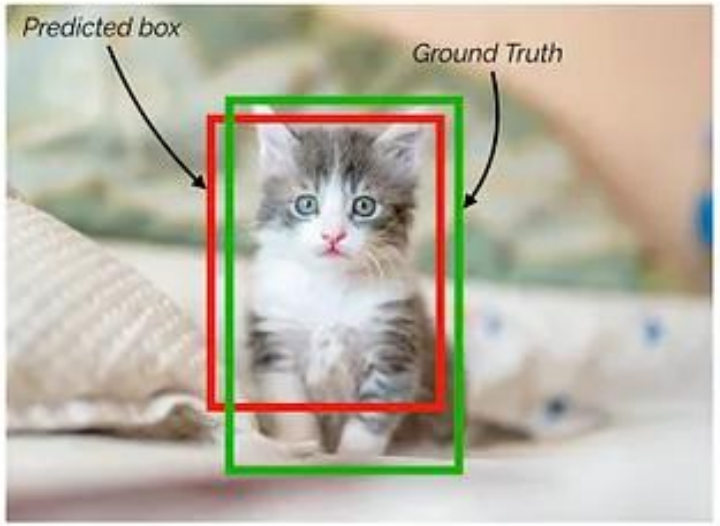
If IoU threshold = 0.5

False Positive (FP)



IoU = ~0.3

True Positive (TP)



IoU = ~0.7

https://towardsdatascience.com/map-mean-average-precision-might-confuse-you-5956f1bfa9e2

Determining Results from IoU

Now coming to ***false negative***.

This is the case when the algorithm does **not detect anything** when it should be detecting. For example, there is a dog in the image but the algorithm does not draw any bounding box around it.

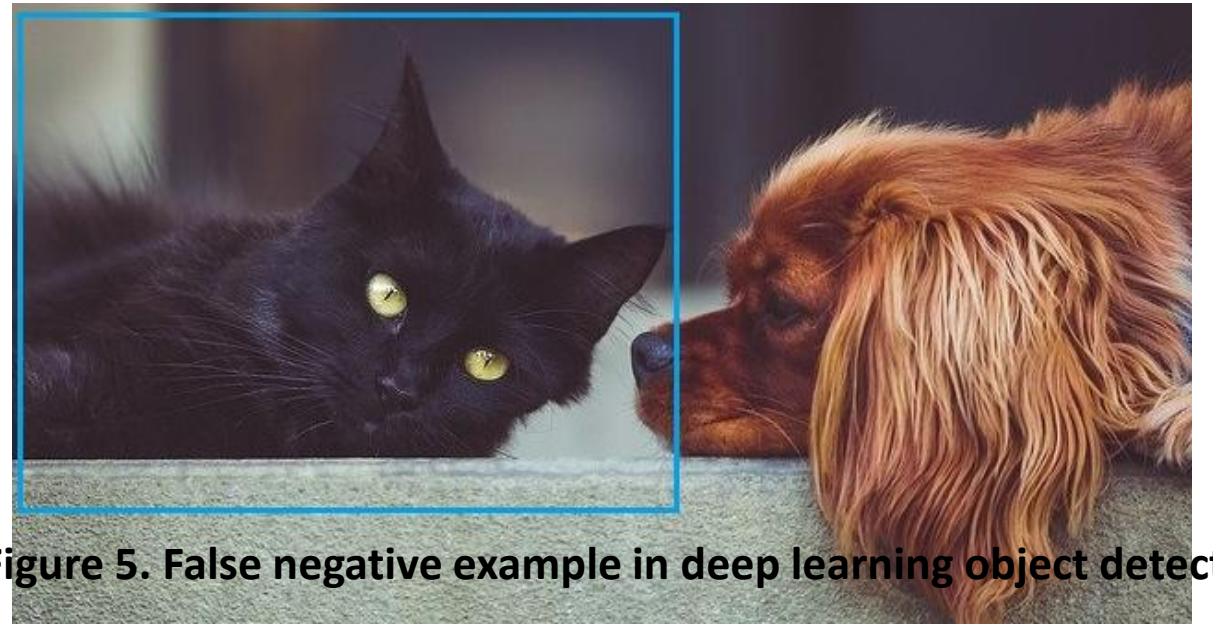
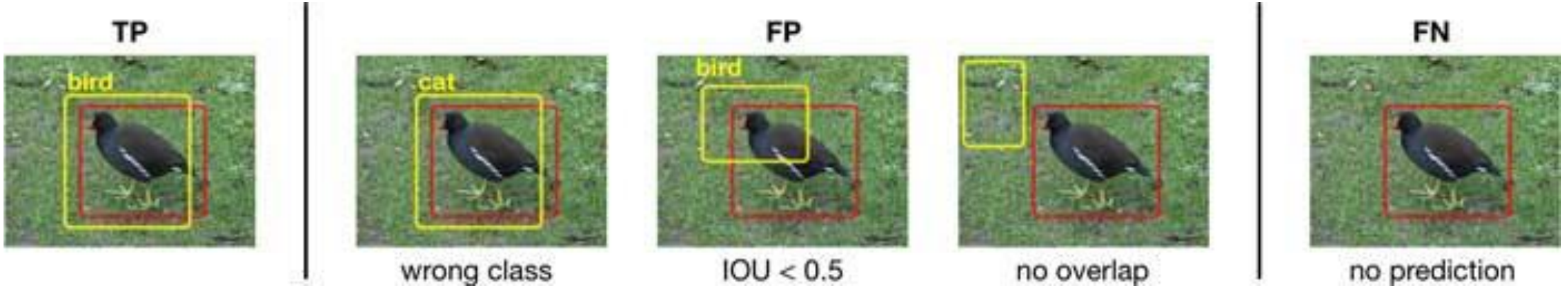


Figure 5. False negative example in deep learning object detection.

Determining Results from IoU

False positive:



Wrong class: this means that we are drawing a bounding box around an object but classifying it as another object

Matching rule

Matching rule (quy tắc ghép nối) → Xác định True Positive (TP), False Positive (FP) khi có nhiều hộp dự đoán và nhiều hộp groud truth trong một ảnh.

Nguyên tắc cốt lõi
Mỗi ground truth chỉ được khớp với tối đa một hộp dự đoán (prediction).
Và hộp dự đoán đó phải là **hộp có IoU cao nhất** trong số các dự đoán có cùng nhãn và cùng ảnh.

Matching rule

Giả sử trong 1 ảnh:

Có **M ground truth boxes (GT_1...GT_m)**

Có **N predicted boxes (P_1...P_n)**

Bước 1 — Lọc theo nhãn

Xét riêng từng **lớp nhãn (class)**.
Ví dụ: chỉ xét “person” hoặc “car” riêng biệt.

Bước 2 — Sắp xếp các dự đoán

Sắp xếp tất cả **predicted boxes** của lớp đó **theo confidence giảm dần**.

Matching rule

Bước 3 — Duyệt từng dự đoán theo thứ tự

Đối với từng hộp dự đoán P_i:

Tính **IoU** giữa $P_i$và **tất cả các ground truth** cùng lớp trong ảnh.

Chọn hộp thật $G_j$có **IoU cao nhất**.

Nếu:

- $IoU(P_i, G_j) \geq 0.5$)hoặc ngưỡng khác tùy quy định), **và**
- G_j**chưa bị ghép trước đó**,
  $\Longrightarrow P_i$là **True Positive (TP)**, đánh dấu $G_j$là "đã được khớp".

Ngược lại $\Longrightarrow P_i$là **False Positive (FP)**.

Matching rule

Ground truths (GT):

ID	Label	Vị trí
G1	person	—
G2	person	—

Predictions:

ID	Label	Confidence	IoU(G1)	IoU(G2)
P1	person	0.95	0.8	0.4
P2	person	0.90	0.7	0.2
P3	person	0.85	0.3	0.6

Matching rule

Bài tập xác định TP, FP, Precision và Recall:

Prediction	Confidence	G1	G2
P1	0.90	0.8	0.3
P2	0.87	0.4	0.65
P3	0.85	0.2	0.55
P4	0.80	0.7	0.3

Average Precision

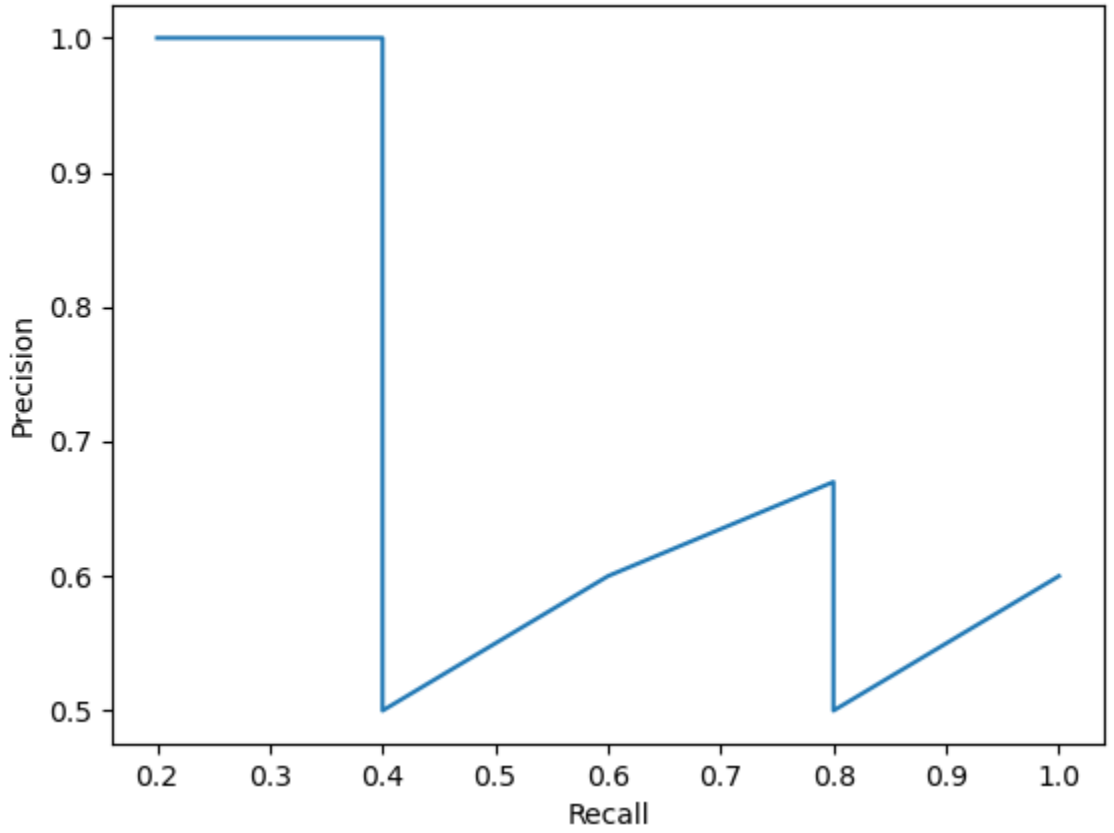
The concept of the ***average precision*** evaluation metric is mainly related to the **PASCAL VOC competitive dataset**.

We need to consider any prediction with an IoU of more than or equal to 0.5 as a true positive.

Average Precision

An example of precision and recall values

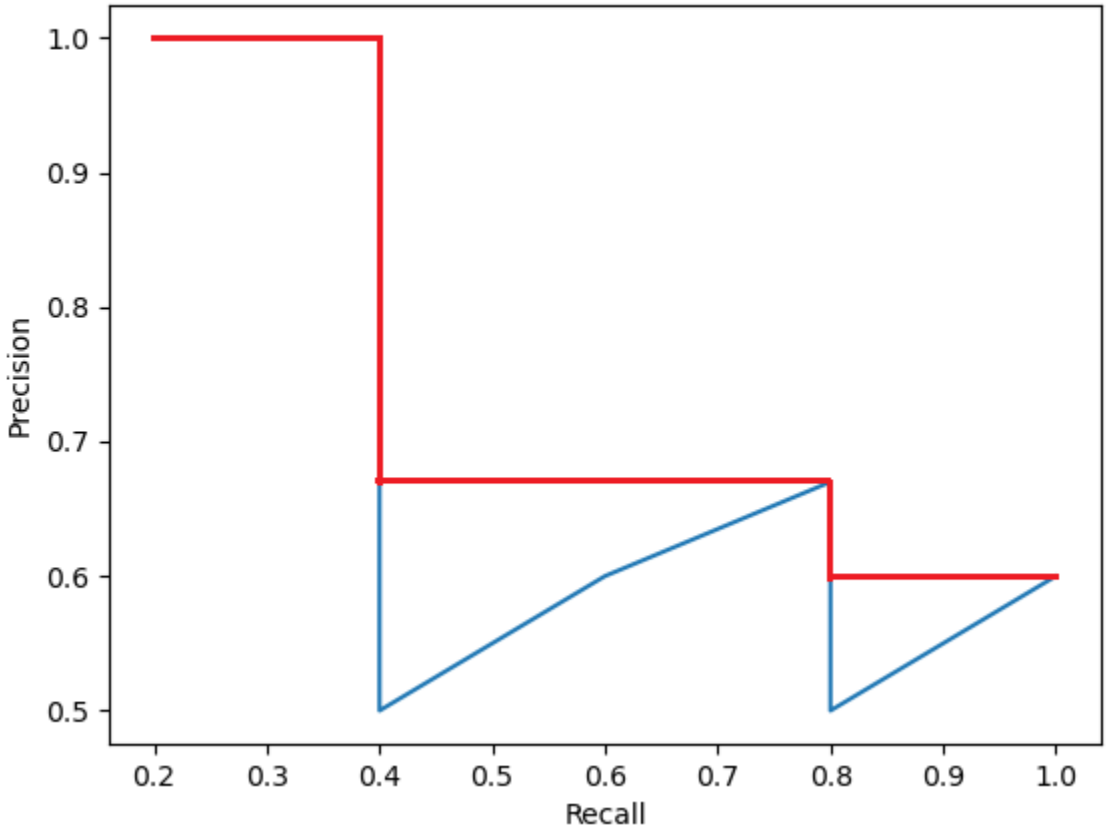
Precision	Recall
1.0	0.2
1.0	0.4
0.7	0.4
0.6	0.4
0.5	0.4
0.6	0.6
0.67	0.8
0.6	0.8
0.5	0.8
0.6	1.0



Average Precision

To get the Average Precision (AP), we need to find the area under the precision vs. recall curve.

But most often, we need to make the lines a bit **smoother** before calculating the area under the graph.

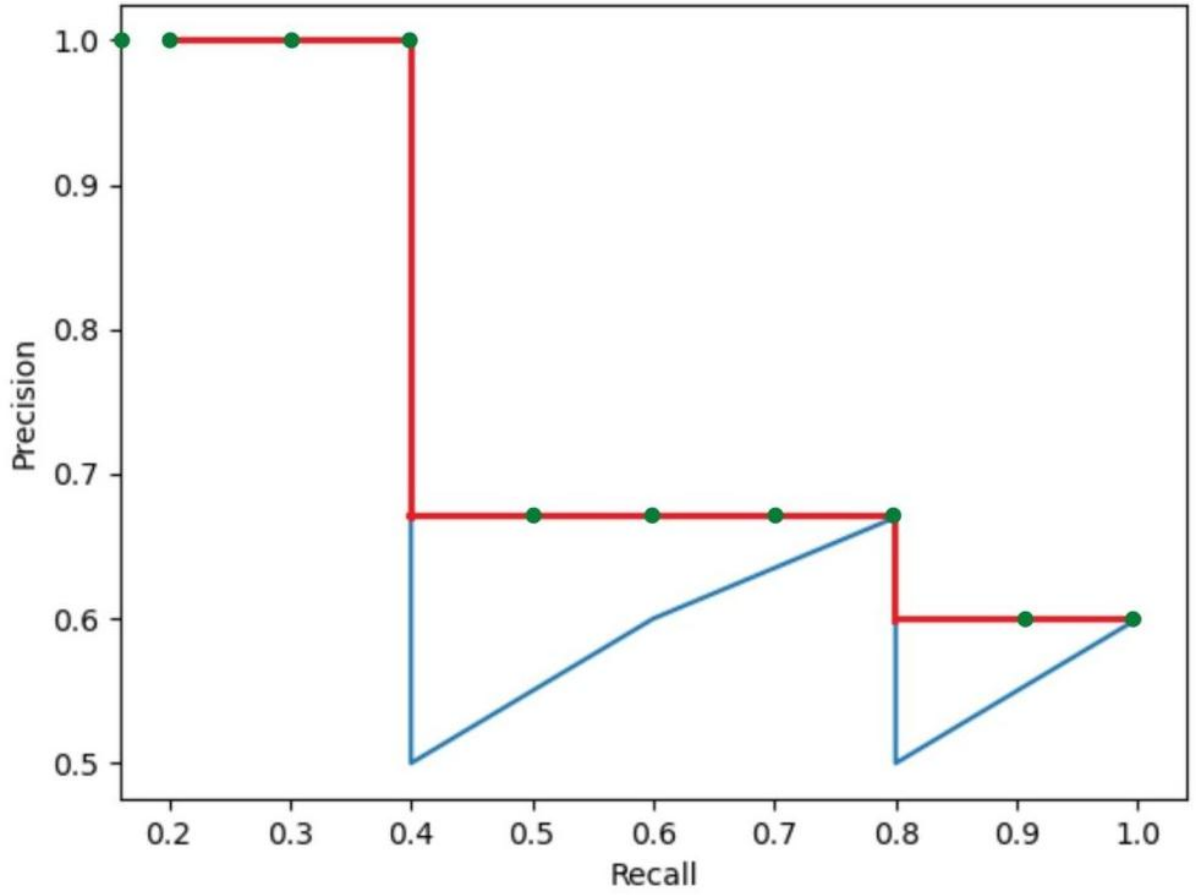


Average Precision

Calculating Average Precision According to PASCAL VOC Challenge

Now, what is interpolated AP?

It is the AP that is defined over a set of equally spaced recall values, from 0.0, 0.1, …, 1.0. We know that the precision and recall values are within the range of 0.0 to 1.0. So, we need to calculate AP for 11 equally spaced values of recall.



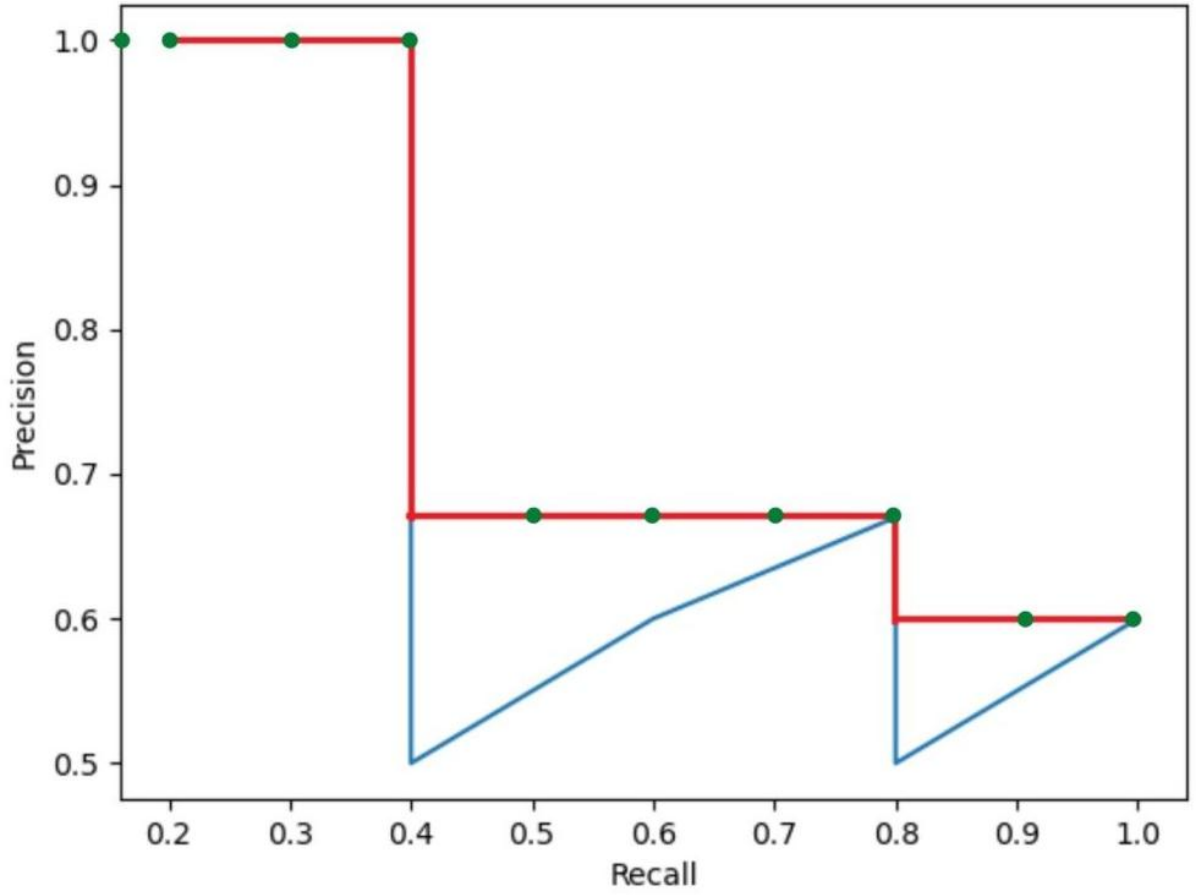
Average Precision

Calculating Average Precision According to PASCAL VOC Challenge

Now, what is interpolated AP?

$$AP = \frac{1}{11} \sum_{r \in \{0, 0.1, \ldots, 1.0\}} p_{interp}(r)$$

$$p_{interp}(r) = max_{\tilde{r}:\tilde{r} \geq r} p(\tilde{r})$$



Average Precision

Graphically, at each recall level, we replace each precision value with the maximum precision value to the right of that recall level.



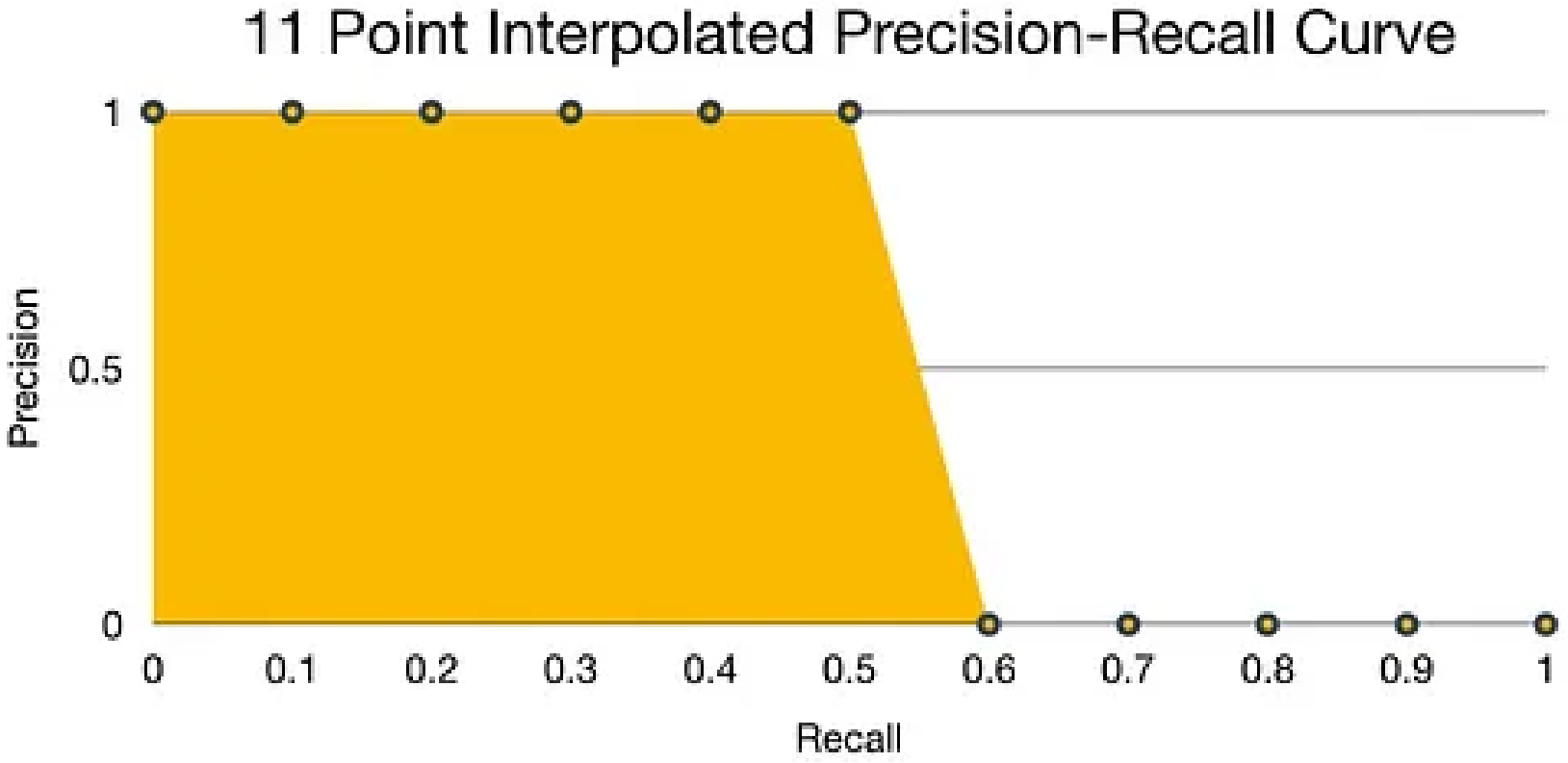
Average Precision

Why do we need an interpolated precision/recall curve to get the proper AP for evaluating our detector?

> *The intention in interpolating the precision/recall curve in this way is to reduce the impact of the "wiggles" in the precision/recall curve, caused by small variations in the ranking of examples. It should be noted that to obtain a high score, a method must have precision at all levels of recall— this penalises methods which retrieve only a subset of examples with high precision (e.g. side views of cars).*
>
> ***The PASCAL Visual Object Classes (VOC) Challenge***

http://host.robots.ox.ac.uk/pascal/VOC/pubs/everingham10.pdf

Average Precision



$$AP = \frac{1}{11} \sum_{0,0.1,0.2,0.3,..0.9,1} (1 * 6) + (0 * 5) = 0.545$$

https://towardsdatascience.com/map-mean-average-precision-might-confuse-you-5956f1bfa9e2

Average Precision

- Với mỗi lớp (class), ta gom tất cả các hộp dự đoán của lớp đó trong toàn bộ dataset,
- Sắp xếp chúng theo confidence giảm dần,
- Sau đó tính Precision–Recall tích lũy khi ta duyệt qua từng hộp (tức là "thêm dần" TP/FP),
- Cuối cùng, lấy diện tích dưới đường cong Precision–Recall (AUC) → đó là AP của lớp đó.

Average Precision

Giả sử lớp "person", có 3 ảnh:

Ảnh	GT (số người thật)	Dự đoán (với confidence và IoU)
A	2	P1 (0.9, TP), P2 (0.8, FP)
B	1	P3 (0.85, TP)
C	1	P4 (0.95, FP), P5 (0.7, TP)

mean Average Precision (mAP)

mean Average Precision (mAP) is the averaged AP over all the object categories.

$$meanAveragePrecision = \frac{\sum_{c \in C} AveragePrecision_c}{|C|}$$

Table 4: Object detection results on PASCAL VOC 2007 *test* set. The performance is measured by mean of Average Precision (mAP, in %).

Method	mAP	areo	bike	bird	boat	bottle	bus	car	cat	chair	cow	table	dog	horse	mbk	prsn	plant	sheep	sofa	train	tv
DenseNet-161 (k=48)	79.9	80.4	85.9	81.2	72.8	68.0	87.1	88.0	88.8	64.0	83.3	75.4	87.5	87.6	81.3	84.2	54.6	83.2	80.2	87.4	77.2
ResNet-101 [16]	76.4	79.8	80.7	76.2	68.3	55.9	85.1	85.3	**89.8**	56.7	**87.8**	69.4	88.3	**88.9**	80.9	78.4	41.7	78.6	79.8	85.3	72.0
ResNeXt-101 (32 × 4d)	80.1	80.2	86.5	79.4	72.5	67.3	86.9	88.6	88.9	64.9	85.0	76.2	87.3	87.8	81.8	84.1	55.5	84.0	79.7	87.9	77.0
DPN-92 (32 × 3d)	**82.5**	**84.4**	**88.5**	**84.6**	**76.5**	**70.7**	**87.9**	**88.8**	89.4	**69.7**	87.0	**76.7**	**89.5**	88.7	**86.0**	**86.1**	**58.4**	**85.0**	**80.4**	**88.2**	**83.1**

COCO: Average Precision

The following 12 metrics are used for characterizing the performance of an object detector on COCO:

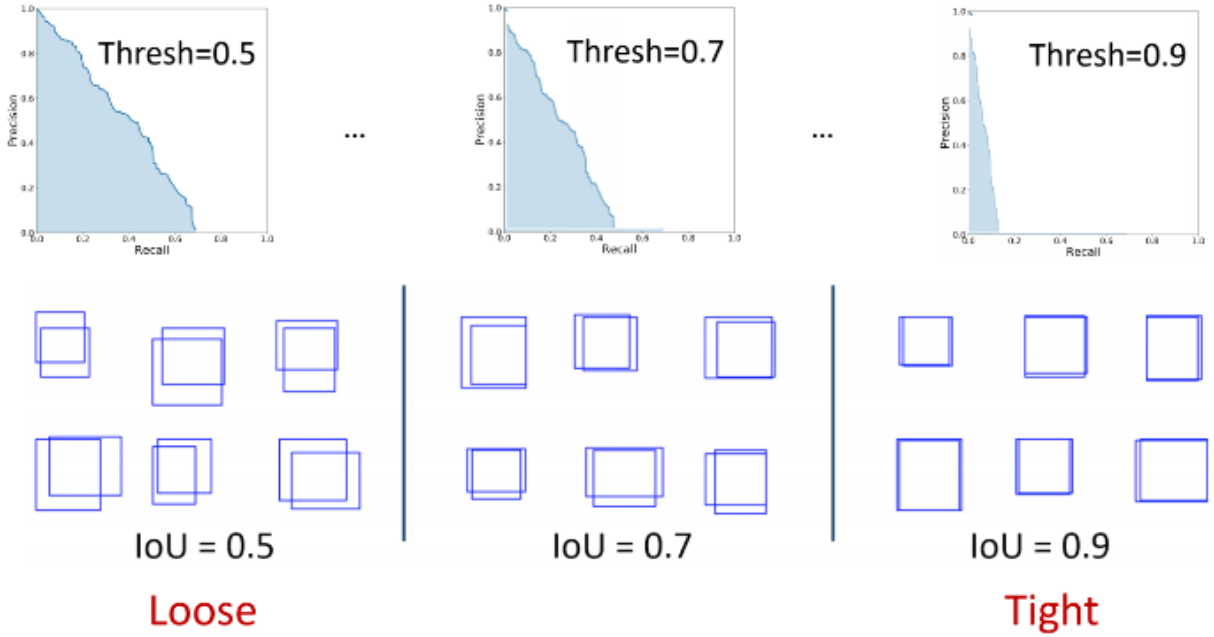
```
Average Precision (AP):
   AP                  % AP at IoU=.50:.05:.95 (primary challenge metric)
   AP^IoU=.50          % AP at IoU=.50 (PASCAL VOC metric)
   AP^IoU=.75          % AP at IoU=.75 (strict metric)
AP Across Scales:
   AP^small            % AP for small objects: area < 32^2
   AP^medium           % AP for medium objects: 32^2 < area < 96^2
   AP^large            % AP for large objects: area > 96^2
Average Recall (AR):
   AR^max=1            % AR given 1 detection per image
   AR^max=10           % AR given 10 detections per image
   AR^max=100          % AR given 100 detections per image
AR Across Scales:
   AR^small            % AR for small objects: area < 32^2
   AR^medium           % AR for medium objects: 32^2 < area < 96^2
   AR^large            % AR for large objects: area > 96^2
```

https://cocodataset.org/#detection-eval

COCO: Average Precision

- *Step 1:* For each class, calculate AP at different IoU thresholds and take their average to get the AP of that class.

$$\text{AP}[\text{class}] = \frac{1}{\#\text{thresolds}} \sum_{\text{iou} \in \text{thresholds}} AP[class, iou]$$



- *Step 2:* Calculate the final AP by averaging the AP over different classes.

$$\text{AP} = \frac{1}{\#\text{classes}} \sum_{\text{class} \in \text{classes}} AP[class]$$

COCO: Average Precision

🎯 **Giả sử:**

- Chúng ta đang đánh giá **1 lớp đối tượng: "Person"**
- Dataset có **3 ảnh**, với **5 Ground Truth (GT)** đối tượng tổng cộng
- Hệ thống phát hiện được **6 Bounding Boxes (Predictions)** với confidence khác nhau

ID	Ảnh	Confidence	IoU cao nhất với GT	Kết quả (nếu threshold=0.5)
P1	A	0.95	0.83	✅ TP
P2	B	0.88	0.72	✅ TP
P3	A	0.80	0.49	❌ FP
P4	C	0.70	0.60	✅ TP
P5	B	0.60	0.40	❌ FP
P6	C	0.55	0.85	✅ TP

COCO: mAP

$$mAP_{\text{COCO}} = \frac{mAP_{0.50} + mAP_{0.55} + \ldots + mAP_{0.95}}{10}$$

https://cocodataset.org/#detection-eval

Why the Average Precision isn't Enough

The source of errors in object detection tasks can be various:

- the model could find the correct location of the bounding box but output it with an incorrect class
- the model could find an object with a correct class but be totally wrong about the location of the bounding box
- the model could miss an object entirely
- and so on

https://towardsdatascience.com/introduction-to-object-detection-model-evaluation-3a789220a9bf

Why the Average Precision isn't Enough

TIDE (Toolbox for Identifying Object Detection Errors):

- **Classification Error:** localized correctly but classified incorrectly
- **Localization Error:** classified correctly but localized incorrectly
- **Both Cls and Loc Error:** classified incorrectly and localized incorrectly
- **Duplicate Detection Error:** would be correct if not for a higher scoring detection
- **Background Error:** detected background as fore-ground
- **Missed GT Error:** All undetected ground truth (false negatives) not already covered by classification or localization error

https://towardsdatascience.com/introduction-to-object-detection-model-evaluation-3a789220a9bf

Why the Average Precision isn't Enough

TIDE (Toolbox for Identifying Object Detection Errors):

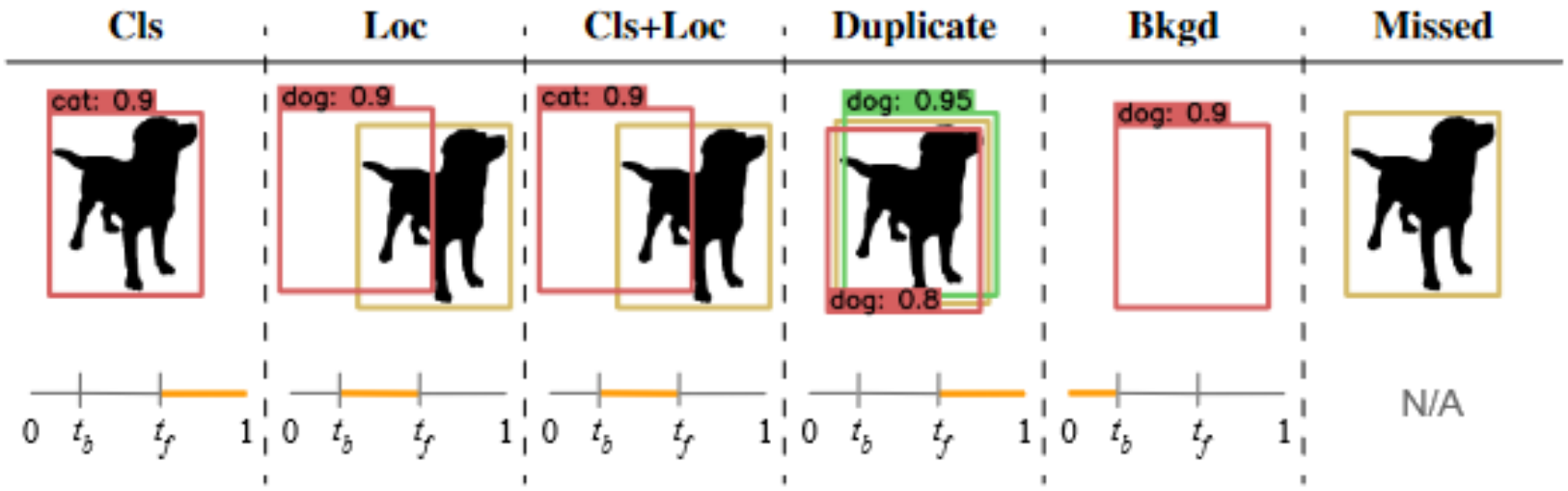
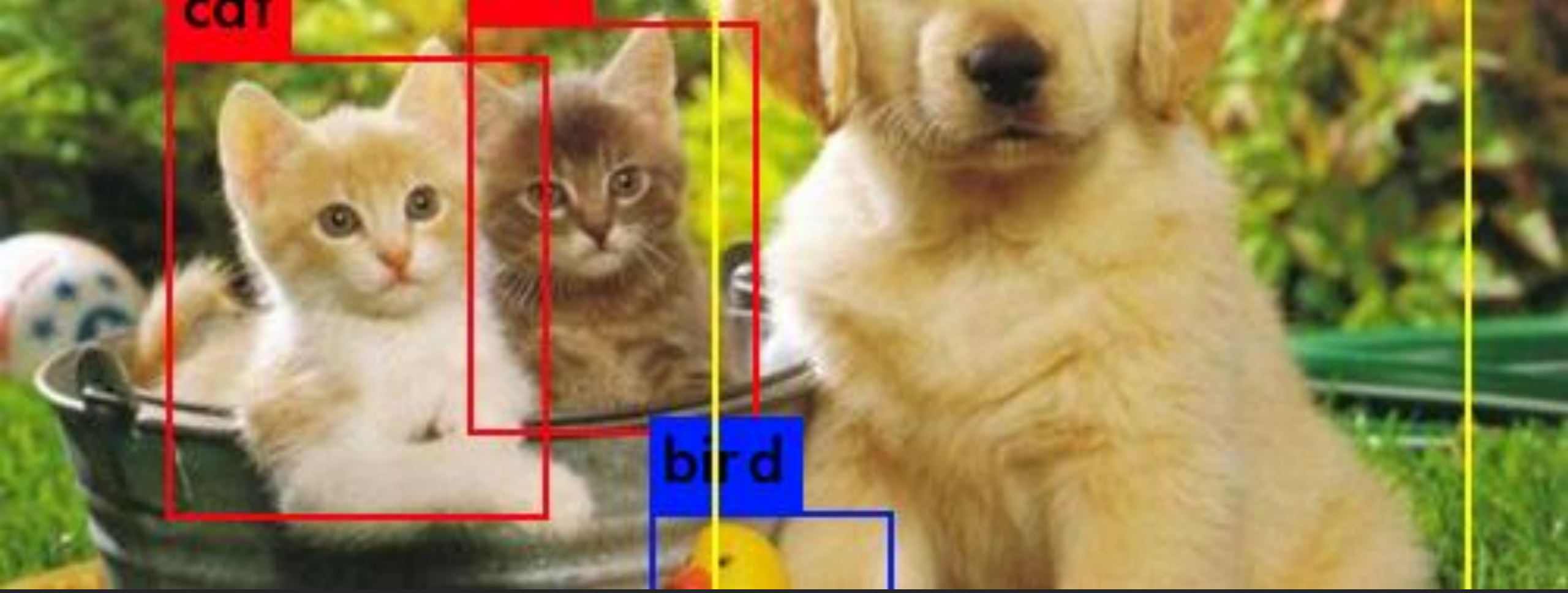


Fig. 1: **Error Type Definitions.** We define 6 error types, illustrated in the *top row*, where box colors are defined as: ■ = false positive detection; ■ = ground truth; ■ = true positive detection. The IoU with ground truth for each error type is indicated by an orange highlight and shown in the *bottom row*.

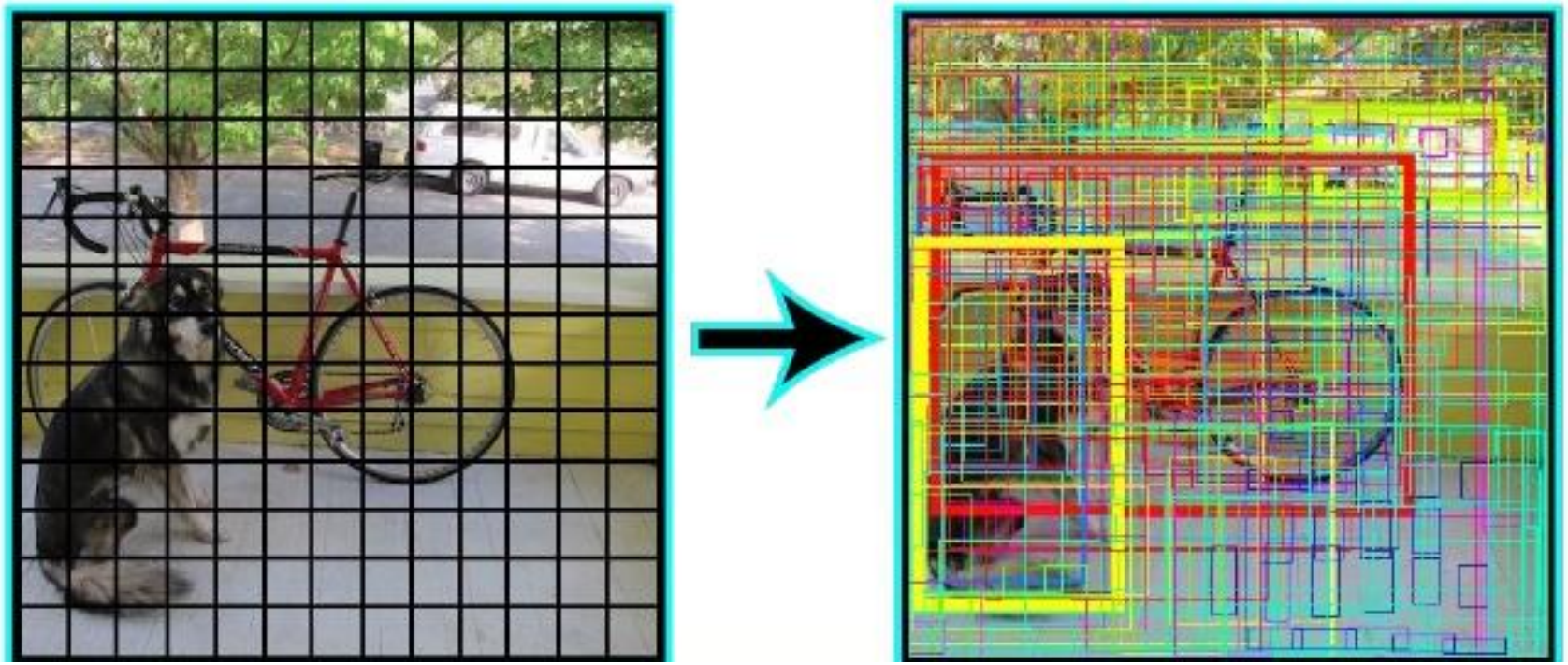
https://towardsdatascience.com/introduction-to-object-detection-model-evaluation-3a789220a9bf



Non-maximum suppression

How To Handle So Many Detections?

- Problem:
  - Running this algorithm at many locations at many scales result with many detections.
- Solution:
  - Need somehow to suppress weaker detections.



Non-maximum suppression (NMS)

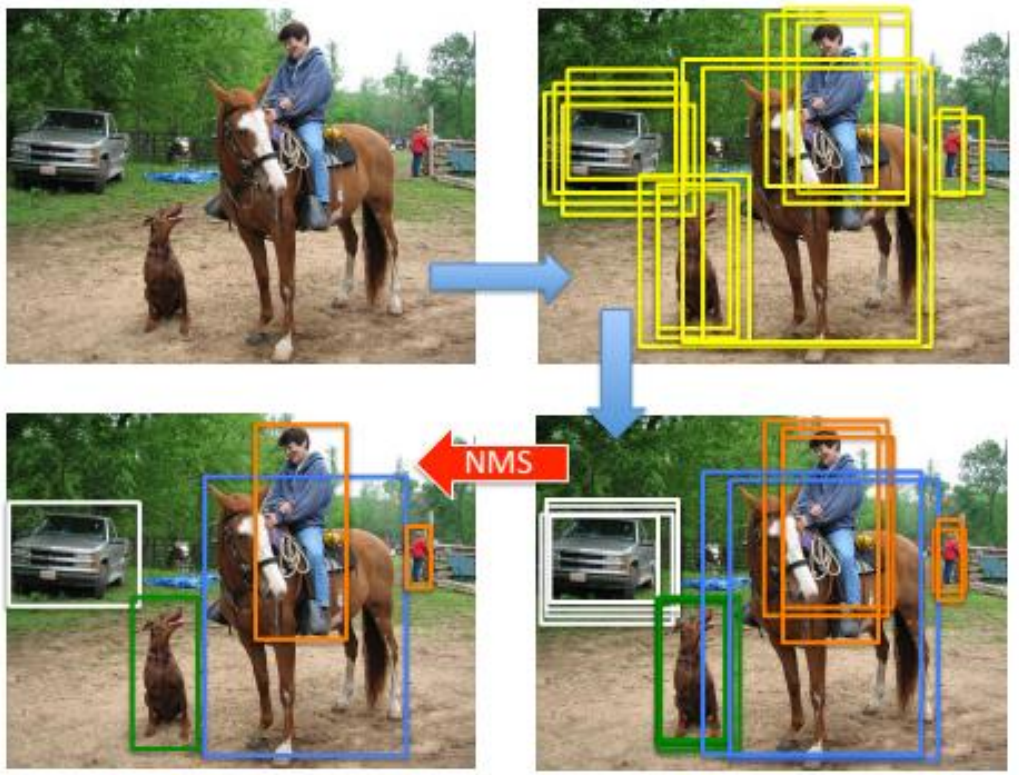


Figure 3. In object detection, first category independent region proposals are generated. These region proposals are then assigned a score for each class label using a classification network and their positions are updated slightly using a regression network. Finally, non-maximum-suppression is applied to obtain detections.

Non-maximum suppression (NMS)

Algorithm 1 Non-Max Suppression

procedure NMS(B,c)
  $B_{nms} \leftarrow \emptyset$ Initialize empty set
  for $b_i \in B$ **do** => Iterate over all the boxes
    $discard \leftarrow$ False Take boolean variable and set it as false. This variable indicates whether b(i) should be kept or discarded
    for $b_j \in B$ **do** Start another loop to compare with b(i)
      if same$(b_i, b_j) > \boldsymbol{\lambda_{nms}}$ **then** If both boxes having same IOU
        if score$(c, b_j) >$ score(c, b_i) **then**
          $discard \leftarrow$ True Compare the scores. If score of b(i) is less than that of b(j), b(i) should be discarded, so set the flag to True.
    if not $discard$ **then** Once b(i) is compared with all other boxes and still the discarded flag is False, then b(i) should be considered. So add it to the final list.
      $B_{nms} \leftarrow B_{nms} \cup b_i$
  return B_{nms} Do the same procedure for remaining boxes and return the final list

pseudo code of NMS

https://towardsdatascience.com/non-maximum-suppression-nms-93ce178e177c

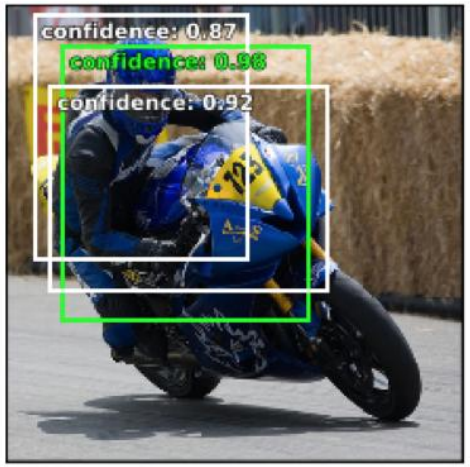
Non-maximum suppression (NMS)

Repeat with next highest confidence prediction until no more boxes are being suppressed

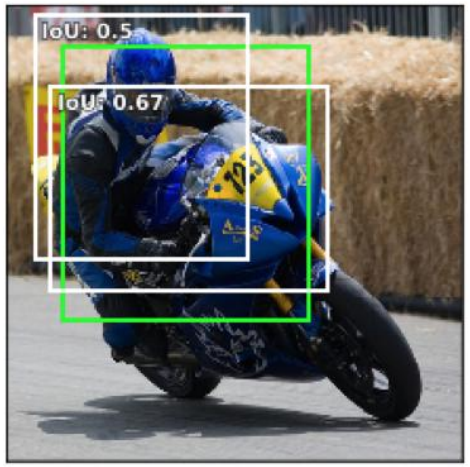
For each class…



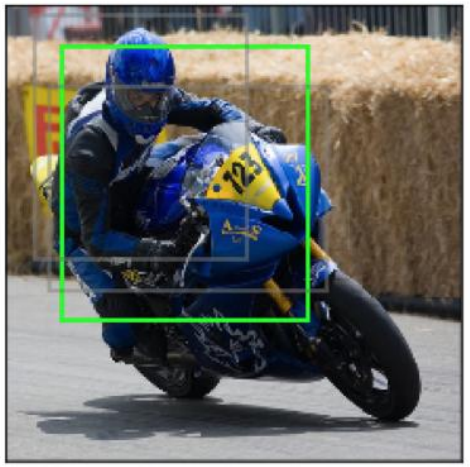
After filtering out low confidence predictions, we may still be left with ***redundant*** **detections**



Select the bounding box prediction with the **highest confidence**



Calculate the IoU between the **selected box** and all remaining predictions



Remove any boxes which have an IoU score above some defined threshold

Non-maximum suppression (NMS)

Now if you observe the algorithm above, the whole filtering process depends on single threshold value.

So selection of **threshold** value is key for **performance** of the model.

However setting this **threshold is tricky**.

Non-maximum suppression (NMS)

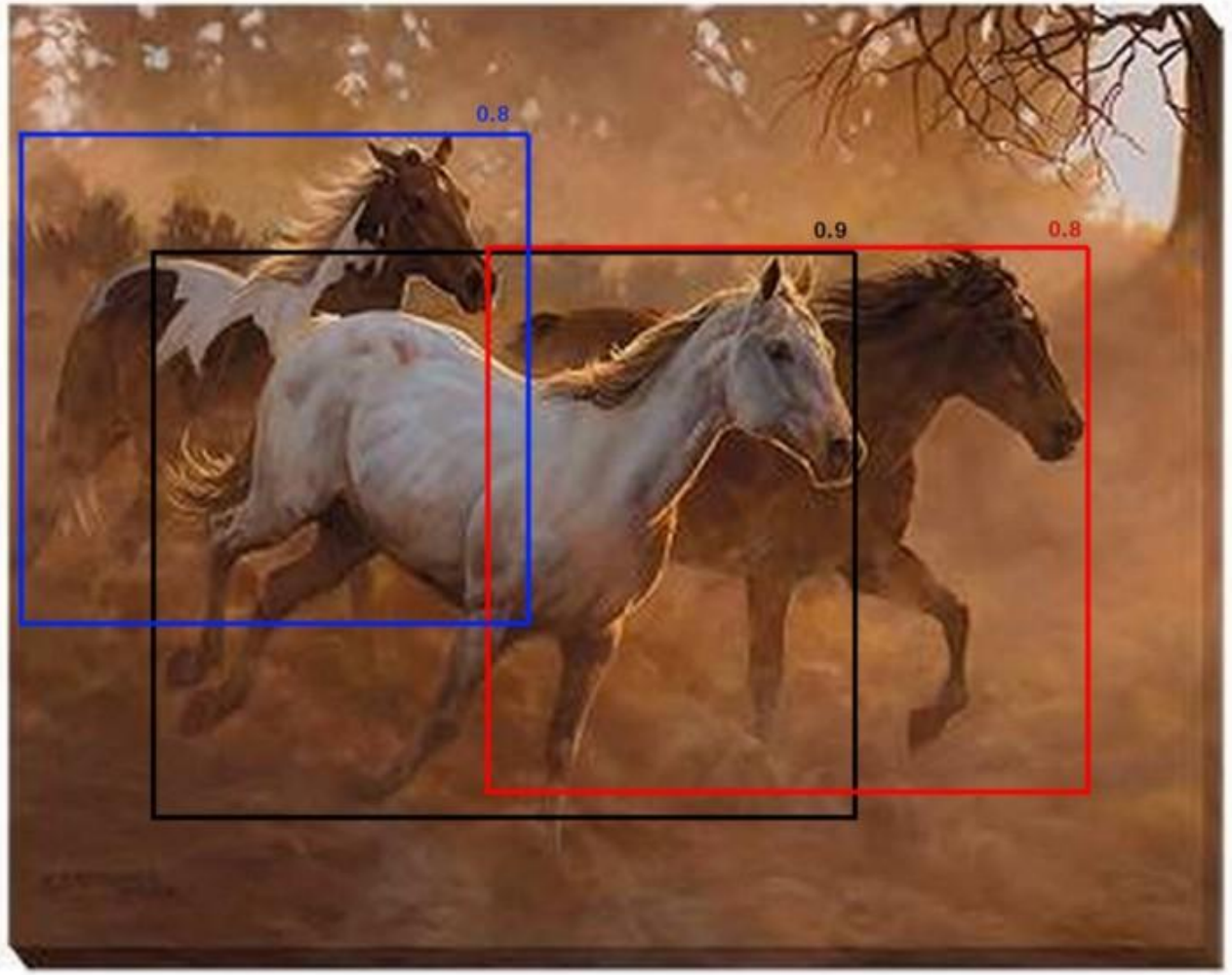
Assume that the overlap threshold N is 0.5.

If there is a proposal with **0.51** IOU and has good confidence score, the box will be **removed** even though the confidence is higher than many other boxes with less IOU. Because of this, if there are two objects side by side, one of them would be eliminated.

A proposal with **0.49** IOU is still **kept** even though its confidence is very low. Of course this is the known issue with any threshold based technique.

Now how do we deal with this? Below is an example of such case. Only the proposal with 0.9 is kept and others will be removed. This reduces the precision of the model.

Non-maximum suppression (NMS)



https://towardsdatascience.com/non-maximum-suppression-nms-93ce178e177c

Soft Non-maximum suppression (Soft-NMS)

Improving Object Detection With One Line of Code

Navaneeth Bodla* Bharat Singh* Rama Chellappa Larry S. Davis
Center For Automation Research, University of Maryland, College Park
{nbodla,bharat,rama,lsd}@umiacs.umd.edu

Abstract

Non-maximum suppression is an integral part of the object detection pipeline. First, it sorts all detection boxes on the basis of their scores. The detection box $\mathcal{M}$ with the maximum score is selected and all other detection boxes with a significant overlap (using a pre-defined threshold) with $\mathcal{M}$ are suppressed. This process is recursively applied on the remaining boxes. As per the design of the algorithm, if an object lies within the predefined overlap threshold, it leads to a miss. To this end, we propose Soft-NMS, an algorithm which decays the detection scores of all other objects as a continuous function of their overlap with $\mathcal{M}$. Hence, no object is eliminated in this process. Soft-NMS obtains consistent improvements for the coco-style mAP metric on standard datasets like PASCAL VOC 2007 (1.7% for both R-FCN and Faster-RCNN) and MS-COCO (1.3% for R-FCN

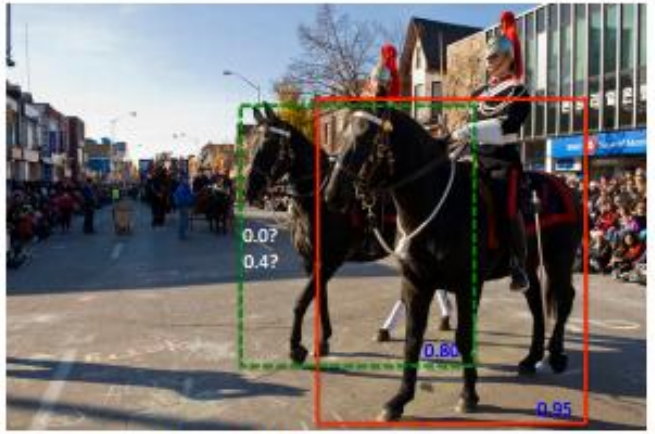


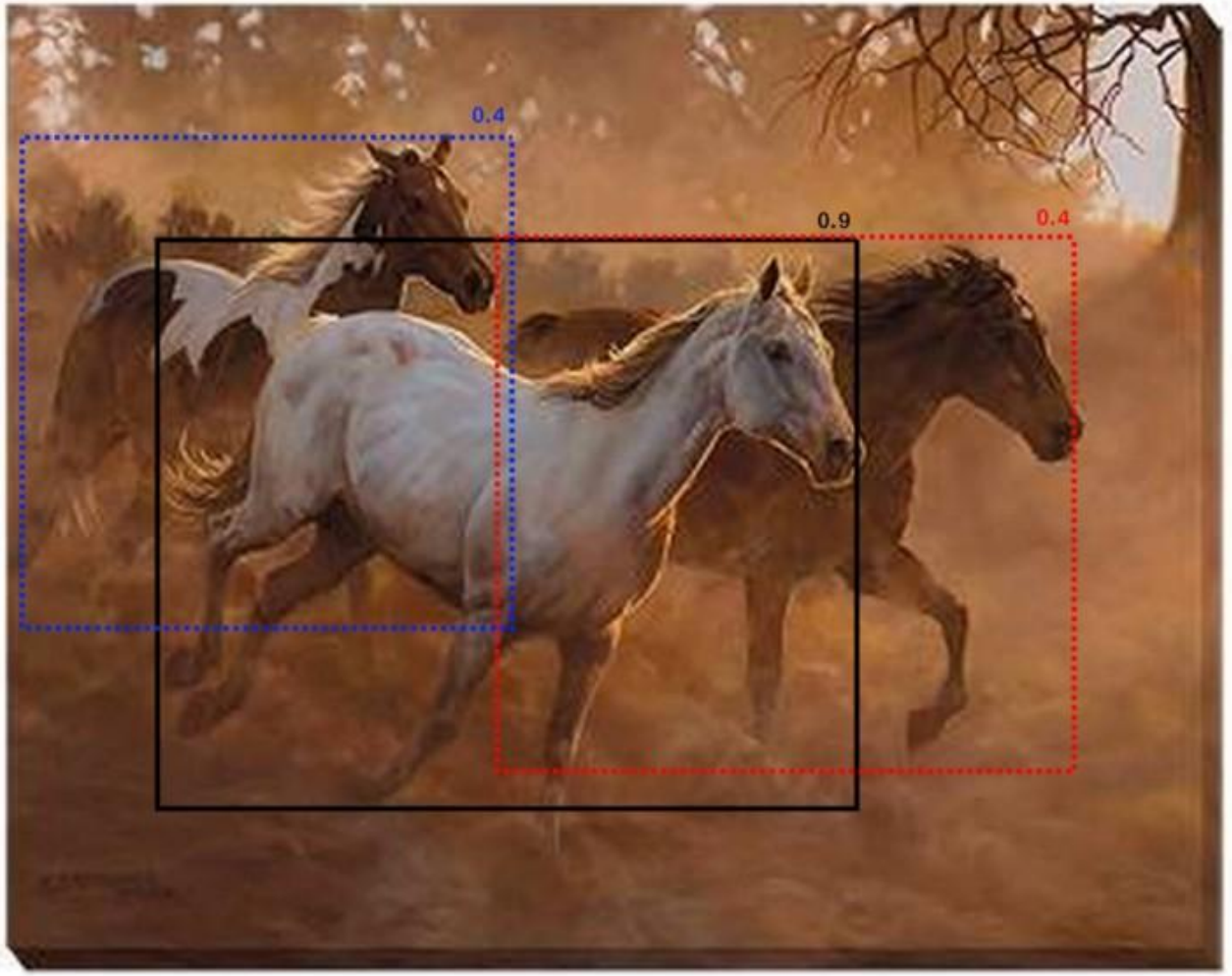
Figure 1. This image has two confident horse detections (shown in red and green) which have a score of 0.95 and 0.8 respectively. The green detection box has a significant overlap with the red one. Is it better to suppress the green box altogether and assign it a score of 0 or a slightly lower score of 0.4?

Soft Non-maximum suppression (Soft-NMS)

The simple yet efficient way to deal with this case is to use **Soft-NMS**. The idea is very simple — “instead of completely removing the proposals with high IOU and high confidence**, reduce the confidences of the proposals proportional to IOU value**”.

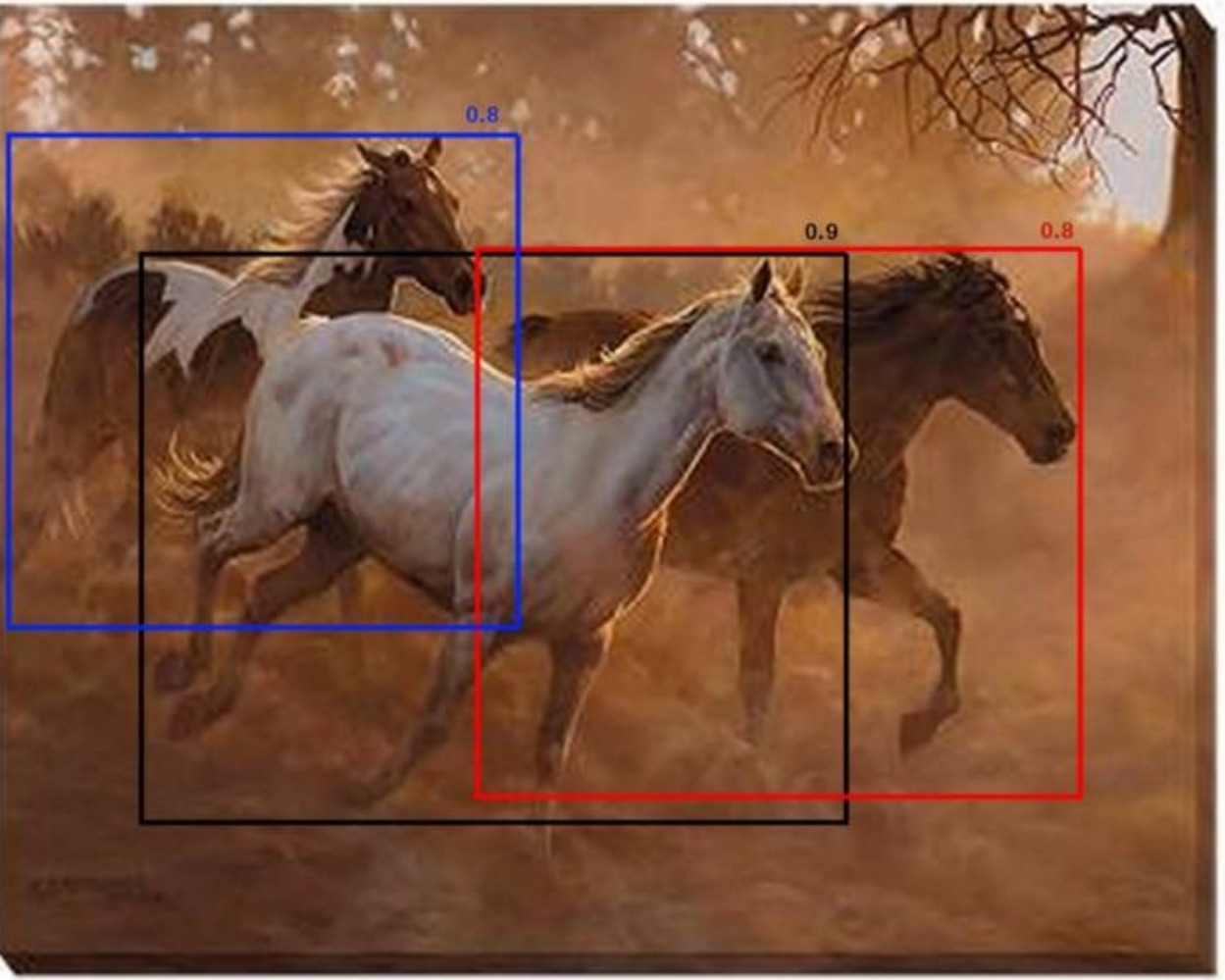
Now let us apply this idea to the above example. instead of completely removing the proposals with 0.8 score, keep the proposals but reduce their score as shown in the figure below.

Soft Non-maximum suppression (Soft-NMS)

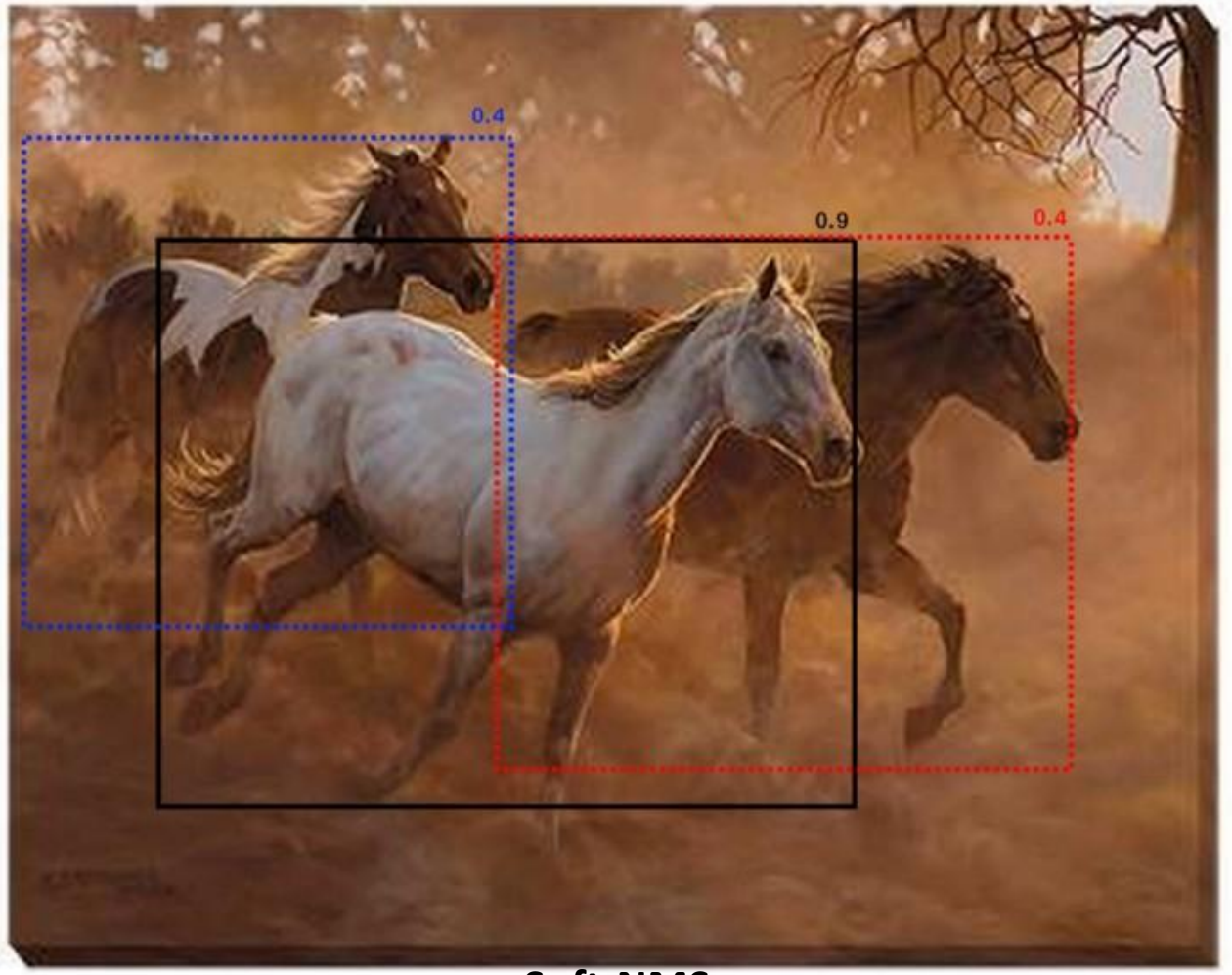


https://towardsdatascience.com/non-maximum-suppression-nms-93ce178e177c

Soft Non-maximum suppression (Soft-NMS)



NMS



Soft-NMS

https://towardsdatascience.com/non-maximum-suppression-nms-93ce178e177c

Soft Non-maximum suppression (Soft-NMS)

As I have mentioned earlier, the scores 0.4 of both the proposals are calculated based on the IOU values. The score calculation is as follows

$$s_i = \begin{cases} s_i, & \text{iou}(\mathcal{M}, b_i) < N_t \\ s_i(1 - \text{iou}(\mathcal{M}, b_i)), & \text{iou}(\mathcal{M}, b_i) \geq N_t \end{cases},$$

Soft-NMS formula https://arxiv.org/pdf/1704.04503.pdf

s_i — score of proposal i,
b_i — box corresponding to proposal i,
M — box corresponding to maximum confidence,
N_t — IOU threshold

The algorithms (NMS and Soft-NMS

Input : $\mathcal{B} = \{b_1, .., b_N\}, \mathcal{S} = \{s_1, .., s_N\}, N_t$
$\mathcal{B}$ is the list of initial detection boxes
$\mathcal{S}$ contains corresponding detection scores
N_t is the NMS threshold

begin
  $\mathcal{D} \leftarrow \{\}$
  while $\mathcal{B} \neq empty$ **do**
    $m \leftarrow$ argmax $\mathcal{S}$
    $\mathcal{M} \leftarrow b_m$
    $\mathcal{D} \leftarrow \mathcal{D} \bigcup \mathcal{M}; \mathcal{B} \leftarrow \mathcal{B} - \mathcal{M}$
    for b_i *in* $\mathcal{B}$ **do**
      if $iou(\mathcal{M}, b_i) \geq N_t$ **then**
        $\mathcal{B} \leftarrow \mathcal{B} - b_i; \mathcal{S} \leftarrow \mathcal{S} - s_i$
      end
      NMS
      $s_i \leftarrow s_i f(iou(\mathcal{M}, b_i))$
      Soft-NMS
    end
  end
  return $\mathcal{D}, \mathcal{S}$
end

https://towardsdatascience.com/non-maximum-suppression-nms-93ce178e177c

Soft Non-maximum suppression (Soft-NMS)

Input: A list of Proposal boxes B, corresponding confidence scores S and overlap threshold N.

Output: A list of filtered proposals D.

Algorithm:

- Select the proposal with highest confidence score, remove it from B and add it to the final proposal list D. (Initially D is empty).
- Now compare this proposal with all the proposals — calculate the IOU (Intersection over Union) of this proposal with every other proposal. If the IOU is greater than the threshold N, remove that proposal from B.
- Again take the proposal with the highest confidence from the remaining proposals in B and remove it from B and add it to D.
- Once again calculate the IOU of this proposal with all the proposals in B and eliminate the boxes which have high IOU than threshold.
- This process is repeated until there are no more proposals left in B.

Soft Non-maximum suppression (Soft-NMS)

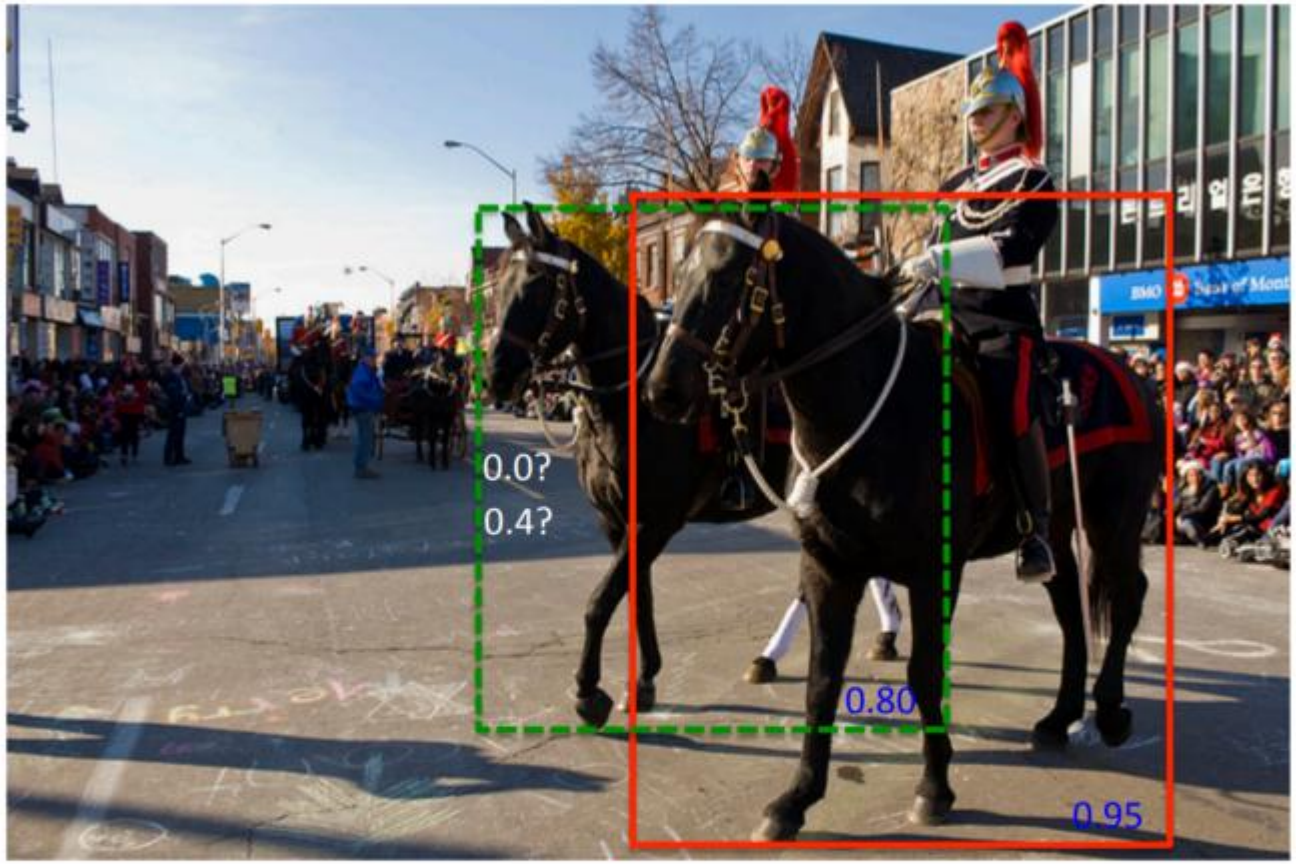


Figure 1. This image has two confident horse detections (shown in red and green) which have a score of 0.95 and 0.8 respectively. The green detection box has a significant overlap with the red one. Is it better to suppress the green box altogether and assign it a score of 0 or a slightly lower score of 0.4?

https://medium.com/@yusuken/object-detction-1-nms-ed00d16fdcf9

Non-maximum suppression (NMS)

We'll perform non-max suppression on **each class separately**.

- Again, the goal here is to remove *redundant* predictions so we shouldn't be concerned if we have two predictions that overlap if one box is describing a person and the other box is describing a bicycle.
- However, if two bounding boxes with high overlap are both describing a person, it's likely that these predictions are describing the same person.

Tài liệu tham khảo

1. https://pyimagesearch.com/2016/11/07/intersection-over-union-iou-for-object-detection/
2. https://debuggercafe.com/evaluation-metrics-for-object-detection/
3. https://machinethink.net/blog/object-detection/
4. https://kharshit.github.io/blog/2019/09/20/evaluation-metrics-for-object-detection-and-segmentation